

INTRODUCTION

1.1 About the Project

In modern democratic societies, elections play a vital role in selecting leaders and representatives. However, traditional voting methods such as paper ballots and manual vote counting have several disadvantages. These methods require significant time, manpower, and resources, and are often prone to human errors, vote manipulation, and delays in result declaration. Additionally, managing voter data and ensuring transparency in the election process can be challenging in manual systems.

To overcome these limitations, the **Online Voting System** is developed as a web-based application that enables secure and efficient digital voting. This project provides a platform where registered users can log in and cast their votes online without the need for physical presence.

The system implements secure authentication mechanisms to verify user identity and ensure that only authorized voters can access the system. Each voter is allowed to vote only once, and the system prevents duplicate voting by maintaining voting status in the database. This improves the fairness and integrity of the election process.

The application also includes functionalities such as nominee management, vote casting, and automatic result generation. An admin dashboard is provided to manage voters, monitor voting activities, and publish results efficiently.

By using this system, the election process becomes faster, more transparent, and easier to manage. It reduces manual work, minimizes errors, enhances security, and provides quick and accurate results. Thus, this project demonstrates how digital technology can effectively solve the problems present in traditional voting systems.

SYSTEM STUDY

2.1 Existing System

In the traditional voting system, elections are conducted using paper ballots or electronic voting machines, where voters are required to physically visit designated polling stations to cast their votes. This process begins with voter verification, where officials check the identity of each voter manually before allowing them to participate in the election. After verification, voters are provided with ballot papers or access to voting machines to record their choice. This entire process requires the involvement of a large number of staff members to manage voter queues, ensure proper identification, distribute materials, and maintain order at polling locations.

Once the voting process is completed, the collected votes are either counted manually or processed through electronic voting machines. Manual counting, in particular, is a time-consuming and labor-intensive task, which may take several hours or even days depending on the number of votes cast. During this phase, there is also a possibility of human errors such as incorrect counting, misplacement of ballots, or delays in compiling the final results. In many small-scale environments such as educational institutions, clubs, and organizations, voting is still conducted using basic manual methods like ballot papers or simple digital tools that lack full automation. These systems do not provide efficient mechanisms to store, manage, and retrieve voter and candidate data. As a result, maintaining accurate records becomes difficult, and tracking voting activity in real time is not possible.

Furthermore, traditional systems face challenges in ensuring transparency and security throughout the election process. Since most operations depend on manual handling, there is a higher risk of errors, data inconsistency, and possible manipulation. Ensuring that each voter votes only once is also difficult without a proper tracking mechanism. Overall, the existing system is time-consuming, resource-intensive, less secure, and not well-suited for modern digital requirements.

2.2 Limitations of Existing System

Although traditional voting systems are widely used, they have several limitations that affect efficiency and reliability. One of the major drawbacks is the requirement for voters to be physically present at polling stations, which can be inconvenient and time-consuming. The vote counting process also takes a significant amount of time, especially when done manually, leading to delays in announcing results.

In addition, these systems depend heavily on human involvement, which increases the chances of errors during voter verification, vote counting, and result calculation. Managing large volumes of voter and candidate data is also difficult in manual systems, making the process less organized and more prone to mistakes. Security is another concern, as there is a possibility of vote manipulation or duplicate voting if proper control mechanisms are not in place. These limitations highlight the need for a more efficient and automated voting solution.

2.3 Proposed System

To overcome the limitations of the existing system, an **Online Voting System** is proposed as a web-based application that enables secure and efficient digital voting. In this system, voters can register and log in using unique credentials, allowing them to participate in the election process from anywhere without the need to visit a polling station.

The system incorporates secure authentication mechanisms to verify the identity of users before granting access. Once authenticated, voters can view the list of nominees and cast their votes through an intuitive interface. Each vote is recorded securely in the database, and the system ensures that every voter can vote only once by maintaining a voting status.

The application also includes an admin dashboard that allows administrators to manage all election-related activities such as adding nominees, registering voters, monitoring voting progress, and generating results. The system automatically calculates the results based on the stored votes, eliminating the need for manual counting.

By automating the entire voting process, the proposed system improves efficiency, reduces

human effort, enhances security, and ensures transparency throughout the election process.

2.4 Advantages of Proposed System

- Faster and more efficient voting and result generation
- Secure authentication with prevention of duplicate voting
- Improved transparency and easy management of election data

2.5 Problem Definition and Description

In traditional voting systems, the election process is often slow, resource-intensive, and prone to errors. The need for physical presence, manual vote counting, and large manpower makes the system inefficient and difficult to manage. Additionally, ensuring security and transparency becomes a major challenge, especially when handling large volumes of data. To address these problems, the Online Voting System is developed as a digital solution that simplifies and automates the election process. The system allows voters to cast their votes online in a secure and convenient manner, while administrators can efficiently manage all election activities through a centralized platform.

By using authentication mechanisms and secure data storage, the system prevents unauthorized access and duplicate voting. It also enables faster vote processing and instant result generation, thereby improving the overall efficiency and reliability of the election process. This project effectively solves the key issues present in traditional voting systems by providing a modern, secure, and user-friendly alternative.

3. SYSTEM ANALYSIS

3.1 PACKAGES SELECTED

Front End : Next.js (React Framework)
Back End : Next.js (API Routes)
Database : MongoDB

3.2 RESOURCES REQUIRED

HARDWARE SPECIFICATION

- Processor : Intel Core i7
- RAM : 4 GB
- Hard Disk : 256 GB
- Keyboard : Standard Keyboard
- Display : Laptop Screen

SOFTWARE SPECIFICATION

- Operating System : Windows OS
- Front End : Next.js
- Back End : Next.js (API Routes)
- Database : MongoDB
- Tool : Visual Studio Code

4. SYSTEM DESIGN

4.1 System Design Overview

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy the specified requirements. It provides a blueprint for the implementation of the system. The main objective of system design is to convert system requirements into a detailed structure that guides the development process.

The **Online Voting System** is designed as a web-based application that allows voters to cast their votes through an online platform. The system consists of several modules including voter authentication, nominee management, vote casting, vote storage, and result generation. These modules work together to ensure that the voting process is secure, transparent, and efficient.

The system design is divided into the following components:

- Architectural Design
- Input and Output Form Design
- Table Design
- Data Flow Diagram
- Database Normalization
- Entity Relationship Diagram
- Data Dictionary

4.2 Architectural Design

Architectural design describes the overall structure of the system and the interaction between different components.

The Online Voting System mainly consists of the following modules:

1. User Authentication Module
2. Voter Management Module
3. Nominee Management Module

4. Vote Casting Module
5. Vote Storage Module
6. Result Calculation Module
7. Admin Dashboard Module

In this system, voters log into the platform using their credentials. After authentication, they can view the list of candidates and cast their vote. The system stores the vote securely in the database and ensures that each voter can vote only once.

The administrator manages the election process through the admin dashboard where they can add nominees, view voter information, monitor votes, and publish election results.

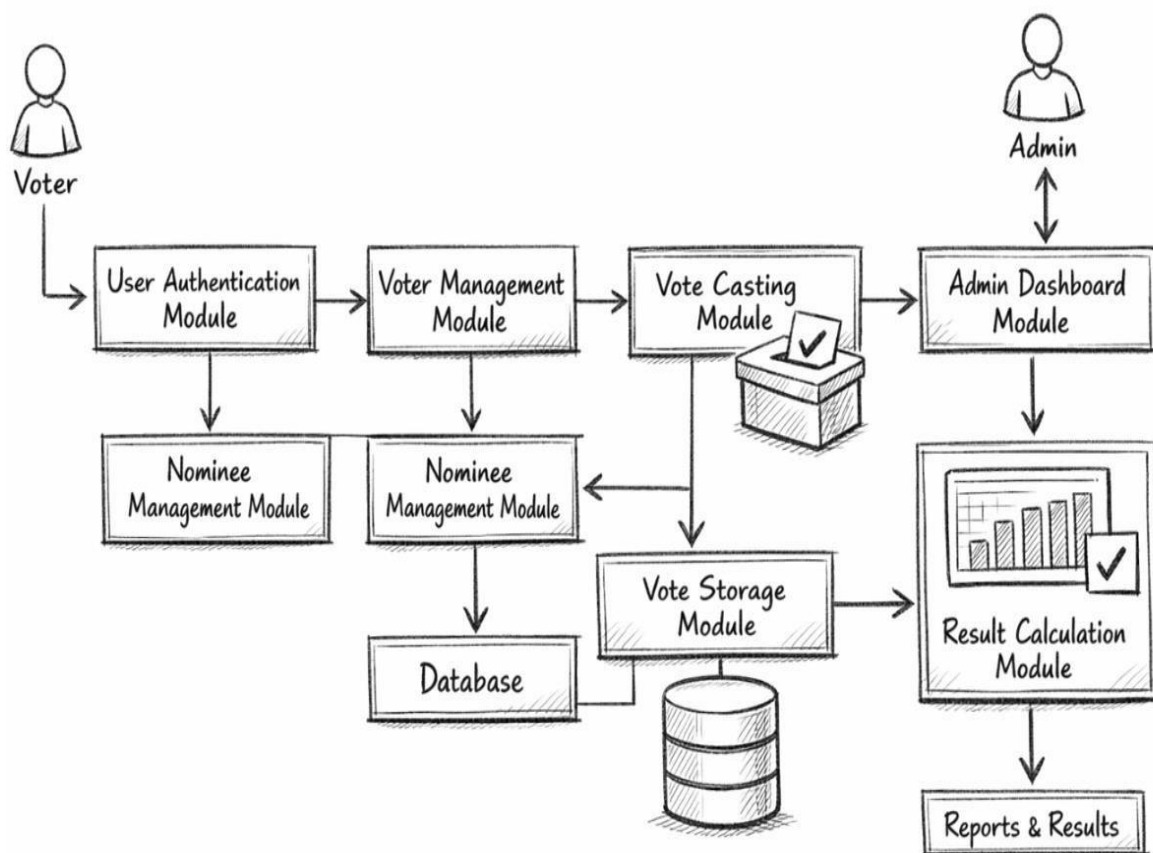


Figure 4.1: System Architecture Diagram

4.2 I/O Form Design

Input and Output design focuses on how data enters the system and how the results are displayed to users.

4.3 Table Design

The image displays seven hand-drawn wireframes for an online voting system, arranged in a 3x3 grid with the bottom-right cell empty. The forms are:

- Login Form:** Includes fields for Username and Password, a Login button, and a link for 'Forgot Password?'.
- Voter Registration Form:** Includes fields for Name, Date of Birth, Voter ID, and Email, along with a Register button.
- Nominee Registration Form:** Includes fields for Name, Party, and Symbol, a Profile Info section, and Register and Clear buttons.
- Nominee Management Form:** Includes a Select Nominee dropdown menu and Add, Update, and Delete buttons.
- Vote Casting Form:** Includes a Select Nominee dropdown menu and a Cast Vote button.
- Election Result Form:** Displays results for four candidates with progress bars and vote counts: Candidate 1 (574), Candidate 2 (292), Candidate 3 (222), and Candidate 4 (204). It includes a View Details button.
- Nominee Management Form (repeated):** A second instance of the Nominee Management Form.
- Vote Casting Form (repeated):** A second instance of the Vote Casting Form.

Table design represents the structure of the database used to store system data. The Online Voting System uses multiple tables to manage election information.

Alright — for a **complete Online Voting System**, you need a few more tables beyond just voters, nominees, and votes. Below is a **full database design in clean table format** (perfect for your project report).

4.3.1. Voters Table

Field Name	Data Type	Key	Description
voter_id	INT	Primary Key	Unique ID for each voter
voter_name	VARCHAR(100)	—	Name of the voter
email	VARCHAR(100)	Unique	Email address
password	VARCHAR(255)	—	Encrypted password
phone	VARCHAR(15)	—	Contact number
has_voted	BOOLEAN	—	Voting status
created_at	DATETIME	—	Registration time

4.3.2. Admin Table

Field Name	Data Type	Key	Description
admin_id	INT	Primary Key	Unique admin ID
admin_name	VARCHAR(100)	—	Admin name
email	VARCHAR(100)	Unique	Admin email
password	VARCHAR(255)	—	Encrypted password

4.3.3. Nominees Table

Field Name	Data Type	Key	Description
nominee_id	INT	Primary Key	Unique nominee ID
nominee_name	VARCHAR(100)	—	Candidate name
party_name	VARCHAR(100)	—	Party name
symbol	VARCHAR(100)	—	Election symbol
vote_count	INT	—	Total votes received

4.3.4. Votes Table

Field Name	Data Type	Key	Description
vote_id	INT	Primary Key	Unique vote ID
voter_id	INT	Foreign Key	References Voters(voter_id)
nominee_id	INT	Foreign Key	References Nominees(nominee_id)
vote_time	DATETIME	—	Time of voting

4.3.5. Election Table

Field Name	Data Type	Key	Description
election_id	INT	Primary Key	Unique election ID
election_name	VARCHAR(100)	—	Name of election
start_date	DATETIME	—	Start time
end_date	DATETIME	—	End time
status	VARCHAR(20)	—	Active / Completed

4.3.6. Results Table

Field Name	Data Type	Key	Description
result_id	INT	Primary Key	Unique result ID
nominee_id	INT	Foreign Key	References Nominees(nominee_id)
total_votes	INT	—	Total votes received
election_id	INT	Foreign Key	References Election(election_id)

4.3.7. Audit Log Table (Optional but Strong for Project)

Field Name	Data Type	Key	Description
log_id	INT	Primary Key	Unique log ID

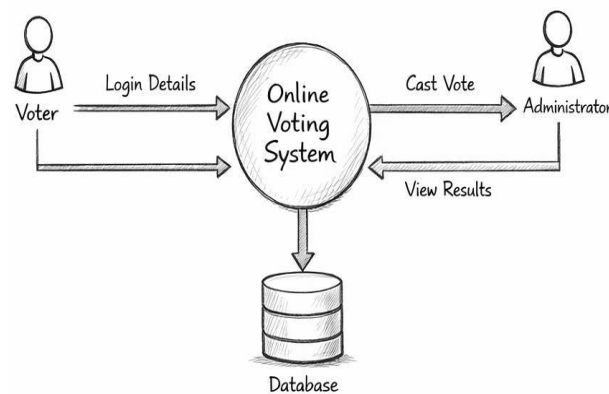
user_id	INT	—	Voter/Admin ID
action	VARCHAR(100)	—	Action performed
timestamp	DATETIME	—	Time of action

4.4 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) shows how data moves through the system and how it is processed by different modules.

DFD Level 0

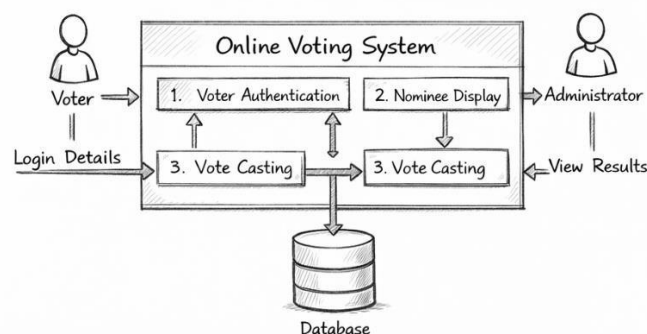
DFD Level 0 - Online Voting System



The Level 0 Data Flow Diagram (DFD) provides an overview of the entire Online Voting System, showing how external users interact with the system. In this diagram, voters provide login details and cast their votes, while the system processes the votes and stores them in the database. The administrator monitors the election process and views the results.

DFD Level 1

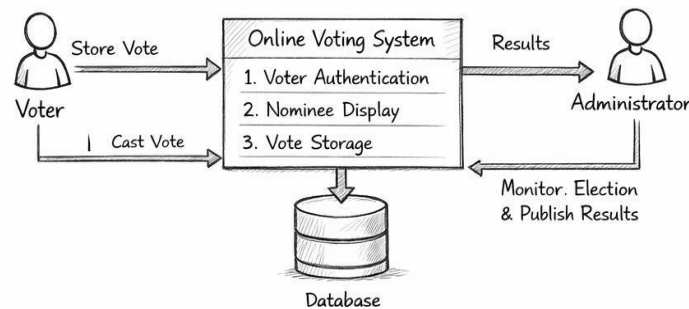
DFD Level 1 - Online Voting System



The Level 1 Data Flow Diagram (DFD) provides a detailed view of the internal

processes of the Online Voting System. It includes processes such as voter authentication, nominee display, vote casting, vote storage, and result calculation, where each process takes input data, performs operations, and produces output results.

DFD Level 2



The Level 2 Data Flow Diagram (DFD) provides a detailed breakdown of the system by dividing each Level 1 process into smaller sub-processes. It includes steps such as voter authentication, nominee display, vote casting with verification, secure vote storage, and result calculation, where votes are counted and results are generated for the administrator.

4.5 Normalization

Database normalization is the process of organizing data in a structured manner to reduce redundancy and improve data integrity. It ensures that the database is efficient, consistent, and free from anomalies such as insertion, deletion, and update issues. The Online Voting System database is designed by applying normalization rules up to the Third Normal Form (3NF).

First Normal Form (1NF)

A table is said to be in First Normal Form if all its attributes contain atomic (indivisible) values and there are no repeating groups or multiple values in a single field.

Example (Before 1NF):

voter_id	voter_name	nominee_names
101	John	A, B, C
102	Mary	A, C

In the above table, the *nominee_names* field contains multiple values, which violates

1NF. After Applying 1NF:

voter_id	voter_name	nominee_name
101	John	A
101	John	B
101	John	C
102	Mary	A
102	Mary	C

Now each field contains only atomic values.

Second Normal Form (2NF)

A table is in Second Normal Form if it is already in 1NF and all non-key attributes are fully dependent on the primary key. This removes partial dependency.

To achieve 2NF, the table is divided into smaller tables:

Voters Table

voter_id	voter_name
101	John
102	Mary

Votes Table

voter_id	nominee_name
101	A
101	B
102	C

Here, voter details are separated, and each non-key attribute depends fully on the primary key.

Third Normal Form (3NF)

A table is in Third Normal Form if it is in 2NF and has no transitive dependency. This means non-key attributes depend only on the primary key and not on other non-key attributes.

To achieve 3NF, further separation is done:

Nominees Table

nominee_id	nominee_name	party_name
-------------------	---------------------	-------------------

Votes Table

vote_id	voter_id	nominee_id
----------------	-----------------	-------------------

Now, all attributes are directly dependent on their respective primary keys, and redundancy is minimized.

Normalization in the Online Voting System ensures that the database structure is well-

organized, reduces data duplication, and improves consistency. By applying 1NF, 2NF, and 3NF, the system achieves better data integrity, efficient storage, and easier maintenance.

4.6 Entity Relationship Diagram (ER Diagram)

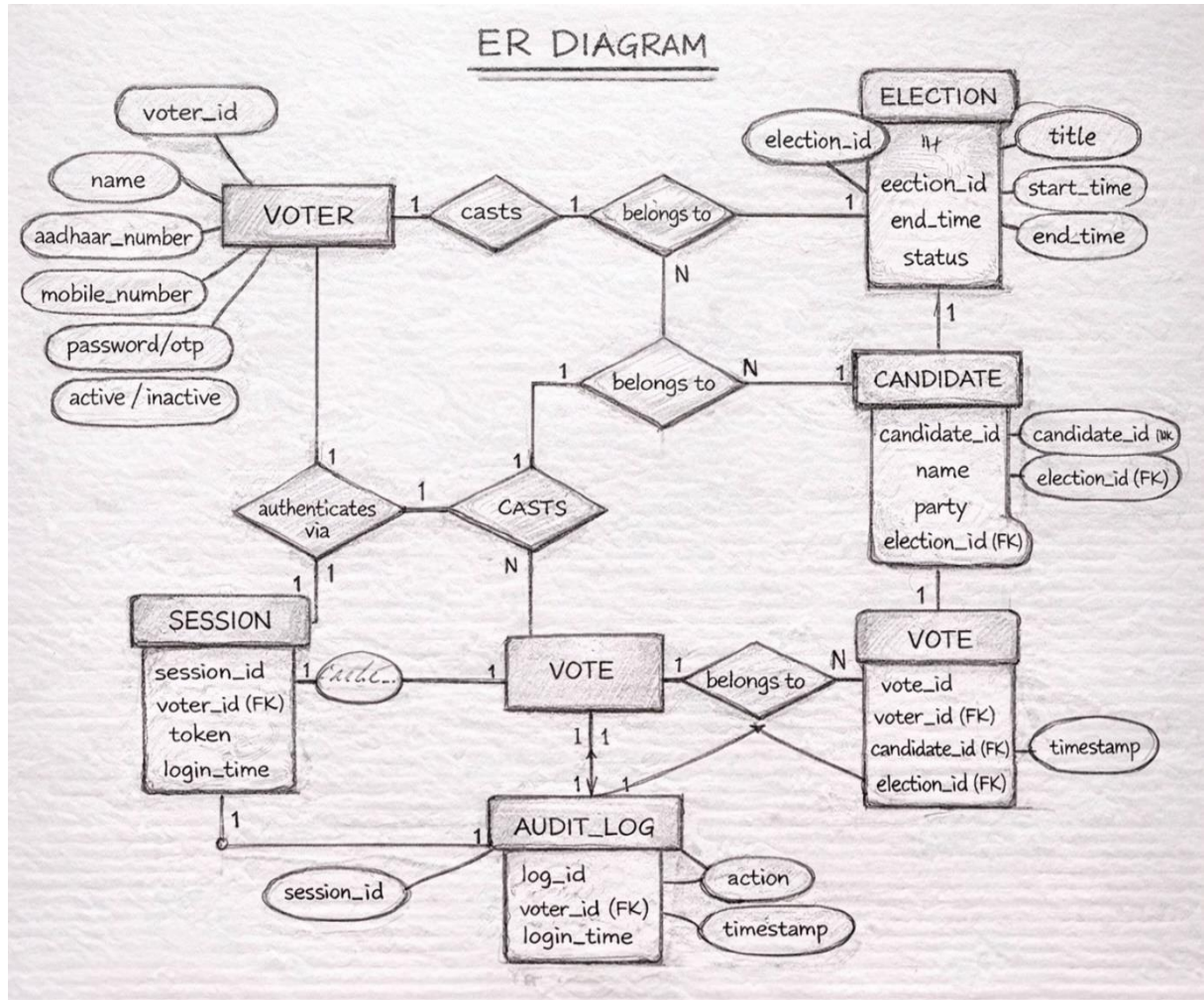


Figure 4.6 Entity Relationship Diagram

4.7 Data Dictionary

The data dictionary describes the meaning and usage of each data element in the system.

Field Name	Description
voter_id	Unique identifier for voter
voter_name	Name of the voter
email	Voter email address
password	Login password
nominee_id	Unique identifier for nominee
nominee_name	Name of the candidate
party_name	Political party name
vote_count	Total votes received
vote_id	Unique identifier for vote
vote_time	Time when vote was cast

5. SYSTEM DEVELOPMENT

System development is the process of implementing the system design using appropriate programming languages, frameworks, and technologies. During this phase, different modules of the system are developed and integrated to perform the required functionalities.

The **Online Voting System** is developed as a web-based application that allows voters to participate in elections through a secure digital platform. The system includes several functional modules that work together to manage the voting process efficiently. These modules handle tasks such as user authentication, nominee management, vote casting, vote storage, and result generation.

The development process ensures that the system is secure, reliable, and easy to use for both voters and administrators.

5.1 Functional Documentation

Functional documentation explains how each module in the system works and how different components interact with each other.

5.1.1 User Authentication Module

The user authentication module is responsible for verifying the identity of voters and administrators before allowing access to the system. Users must log in using their registered credentials such as username and password.

After successful authentication, the system grants access to the voting interface for voters or the admin dashboard for administrators. This module ensures that only authorized users can access the system and participate in the election process.

5.1.2 Voter Registration Module

The voter registration module allows administrators to register voters in the system. Each voter is provided with unique login credentials to access the platform.

The system stores voter details such as name, email, and login information in the database. This module ensures that only registered voters can participate in the election.

5.1.3 Nominee Management Module

The nominee management module allows the administrator to add, update, or remove candidate details from the system.

Each nominee is registered with information such as candidate name, party name, and other relevant details. The nominees added by the administrator are displayed to voters during the voting process.

5.1.4 Vote Casting Module

The vote casting module allows voters to select their preferred candidate and cast their vote through the system interface.

After logging into the system, the voter can view the list of available nominees. The voter selects one candidate and submits the vote. Once the vote is cast, the system records it in the database and prevents the voter from voting again.

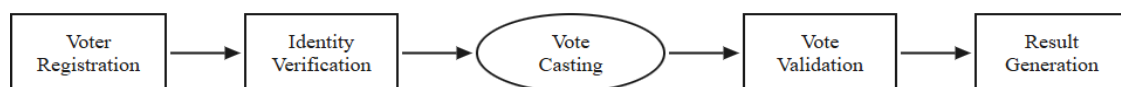


Figure 5.1.4 Vote Casting Pipeline

5.1.5 Vote Storage Module

The vote storage module is responsible for securely storing all voting records in the database.

Each vote is stored along with details such as voter ID, nominee ID, and time of voting. This module ensures data integrity and helps maintain accurate voting records for result calculation.

5.1.6 Result Calculation Module

The Result Calculation Module is responsible for automatically calculating the votes in the system.

In this project, once a voter casts their vote, it is stored securely in the database. The system then retrieves all the stored voting data and calculates the total number of votes received by each nominee.

The result is generated instantly without any manual counting, which reduces errors and saves time. The final results are displayed clearly in the admin dashboard, allowing the administrator to view the winner and overall voting statistics easily.

5.1.7 Admin Dashboard Module

The Admin Dashboard Module acts as the main control panel of the system. It allows the administrator to manage and monitor the entire election process efficiently. Through this module, the admin can:

- Add and manage nominees
- View and manage registered voters
- Monitor voting activity
- View vote counts and statistics
- Publish final election results

The dashboard provides a clear interface for controlling all system operations in one place.

5.1 Special Features of the Language

The Online Voting System is developed using modern web technologies such as Next.js and JavaScript, which provide high performance and scalability.

JavaScript is used as the core programming language to implement application logic and handle dynamic operations in the system.

Next.js is used as the main framework for both frontend and backend development. It helps in building a fast and efficient web application with features like routing, API handling, and optimized performance.

The system uses Next.js API routes to handle backend operations such as user authentication, vote processing, and data management.

6. TESTING

Testing is an important phase in software development where the developed system is evaluated to ensure that it works correctly and meets the specified requirements. The purpose of testing is to identify errors, verify system functionality, and ensure that the system performs efficiently under different conditions.

In the **Online Voting System**, testing was conducted to ensure that all modules of the system work properly and that the voting process is secure and reliable. Different testing methods were applied to verify the accuracy of the voting process, authentication mechanism, and result calculation.

Testing also ensures that the system can handle user interactions smoothly and that voters can cast their votes without any errors. Through proper testing, the reliability and performance of the system are improved before deployment.

The following types of testing were performed during the development of the Online Voting System:

- Unit Testing
- Integration Testing
- System Testing
- Performance Testing
- Security Testing

6.1 Types of Testing Done

Unit Testing

Unit testing is the process of testing individual modules of the system separately to ensure that each module performs its function correctly.

In the Online Voting System, the following modules were tested individually:

- User authentication module
- Voter registration module

- Nominee management module
- Vote casting module
- Result calculation module

Integration Testing

Integration testing is performed to ensure that different modules of the system work properly when combined together.

In the Online Voting System, integration testing was performed to verify the interaction between modules such as:

- Login module and voter dashboard
- Nominee management module and voting interface
- Vote casting module and database storage
- Vote storage module and result calculation module

This testing ensured that data flows correctly between modules without errors.

System Testing

System testing evaluates the complete system to verify that all functionalities operate as expected.

The Online Voting System was tested by simulating the complete election process including voter login, candidate selection, vote casting, and result generation. The system successfully recorded votes and generated results accurately.

Performance Testing

Performance testing is used to evaluate how the system performs under different conditions.

The system was tested to measure factors such as:

- Response time of the application
- Database performance
- Handling multiple users

The system maintained stable performance during testing and was able to handle voting operations efficiently.

Security Testing

Security testing is very important for an online voting system to ensure that voting data

remains safe and protected.

The following security checks were performed:

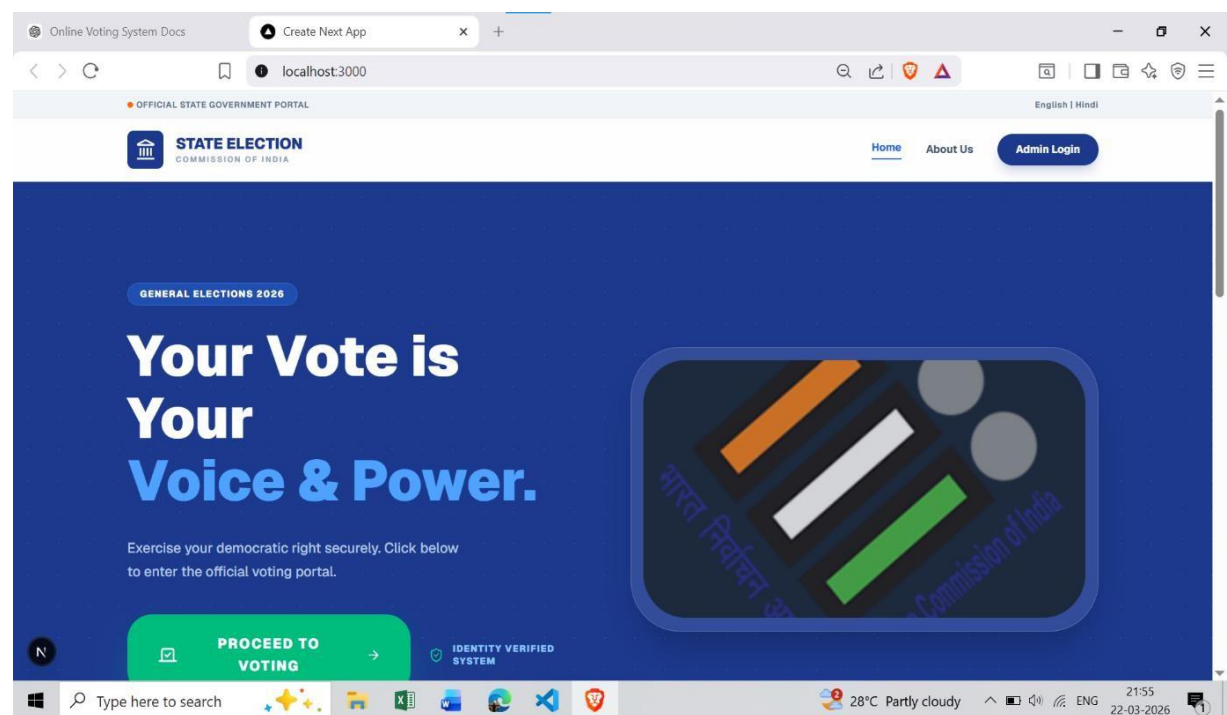
- Verification of user authentication
- Prevention of duplicate voting
- Secure storage of voting data
- Protection against unauthorized access

These security measures ensure that the voting process remains transparent and trustworthy.

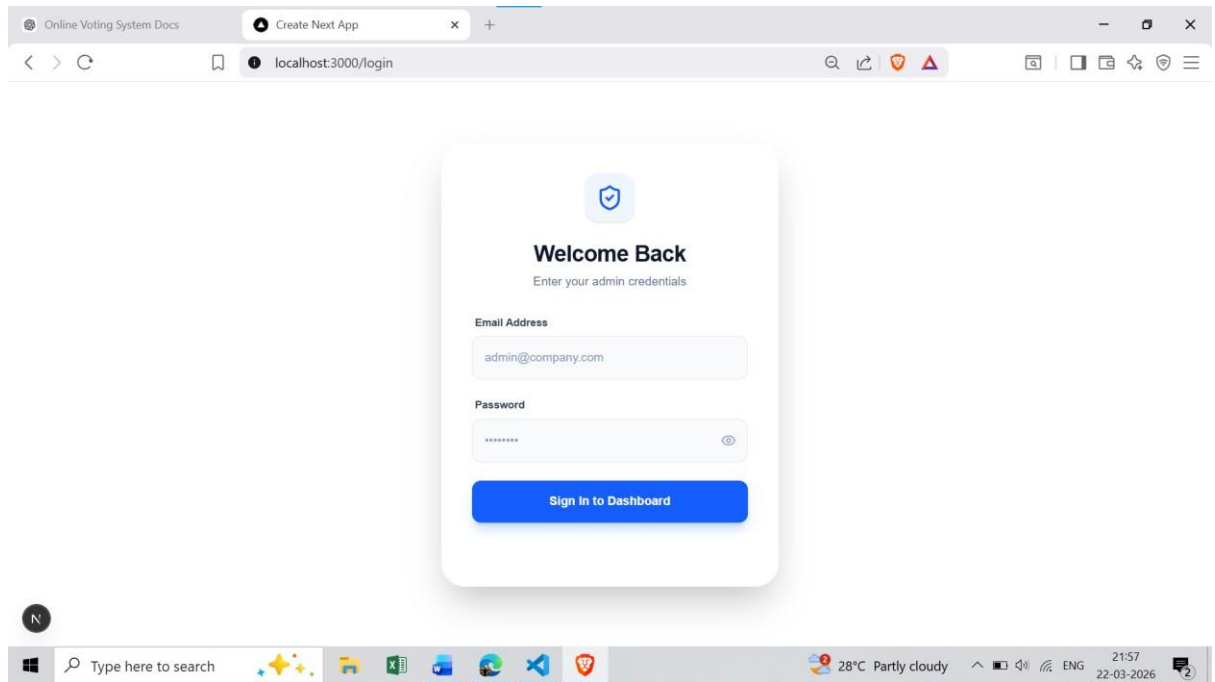
Test Result:

- The system successfully allowed registered users to log in and cast their votes without errors.
- Each voter was able to vote only once, and duplicate voting was effectively prevented.
- Votes were stored correctly in the database and the result calculation was accurate.
- The system performed efficiently with smooth user interaction and quick response time.

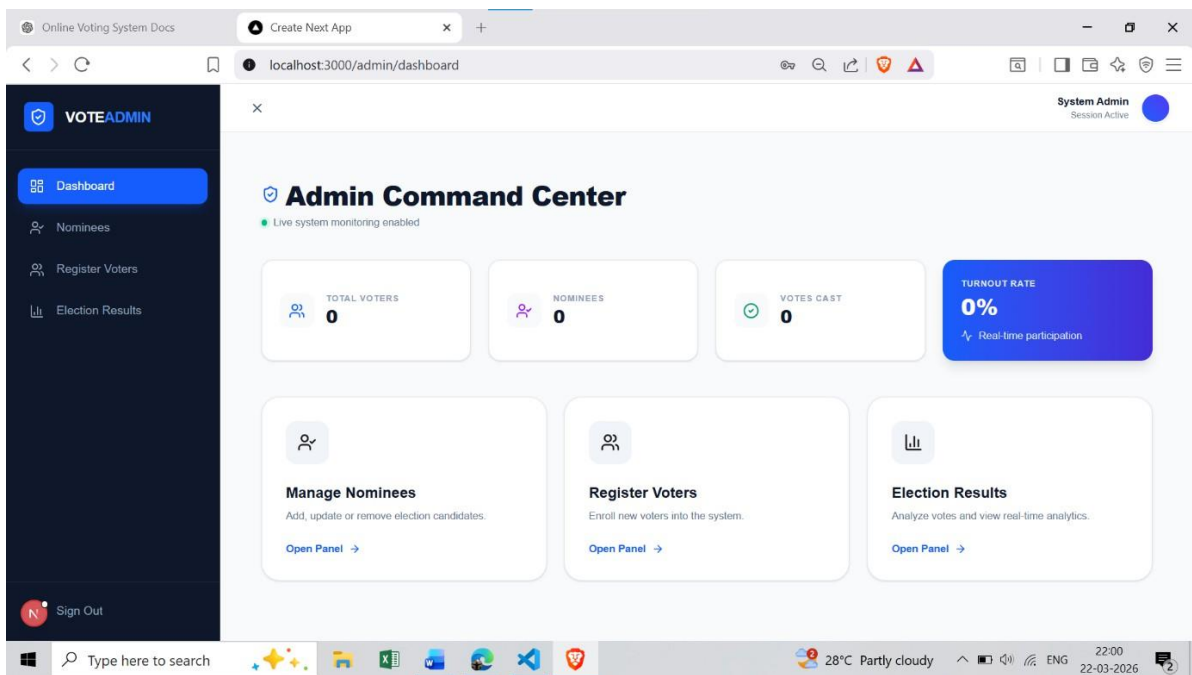
SCREENSHOTS HERE:



6.1 The Online Voting System Launching Page



6.1 The Admin of Login for the Online Voting System



6.2 The rough counting of voting in Admin login

The screenshot shows a web browser window with the URL `localhost:3000/admin/register-voters`. The page is titled "Election Commission Portal" and "Digital Voter Registration System". It features a dark sidebar with navigation links: Dashboard, Nominees, Register Voters (highlighted), and Election Results. A "Sign Out" button is at the bottom of the sidebar. The main content area has a header with the text "Your vote is your voice. Register today and strengthen democracy." Below this is a registration form with the following fields: Full Name (with a placeholder "Enter your full name"), Phone Number (with a placeholder "10 digit mobile number"), Aadhar Number (with a placeholder "12 digit Aadhar"), Date of Birth (with a placeholder "dd-mm-yyyy"), Gender (a dropdown menu currently showing "Male"), and Address (with a placeholder "House number, street, city, pincode"). A blue button labeled "Generate Voter ID >" is at the bottom of the form. The browser's taskbar at the bottom shows the Windows logo, a search bar, and various application icons. The system tray on the right displays the date and time as 22:04 on 22-03-2026, along with weather information (28°C Partly cloudy) and language settings (ENG).

6.3 SIR form in the Admin Login

The screenshot shows a web browser window with the URL `localhost:3000/admin/fetched-votes`. The page is titled "Election Audit Dashboard" with the subtitle "Monitor and verify all votes in real-time". A "Refresh" button is in the top right corner. The dashboard includes a "TOTAL VOTES" section showing a count of 0. Below this is a search bar with the placeholder text "Search by voter name or voter ID..." and a "0 record(s)" indicator. A table with the following headers is displayed: VOTER, VOTER ID, NOMINEE, and DATE. The table body is empty, and a message "No voting records found" is centered below the table. The dark sidebar on the left contains navigation links: Dashboard, Nominees, Register Voters, and Election Results (highlighted). A "Sign Out" button is at the bottom of the sidebar. The browser's taskbar at the bottom shows the Windows logo, a search bar, and various application icons. The system tray on the right displays the date and time as 22:05 on 22-03-2026, along with weather information (28°C Partly cloudy) and language settings (ENG).

6.5 The Realtime Counting the Election Results in the Admin Login

7.USER MANUAL:

The user manual provides detailed instructions on how to install, configure, and operate the Online Voting System. It helps users understand the system requirements, installation process, and how to use the application effectively.

The Online Voting System is designed to be simple, secure, and user-friendly. It allows voters to log in, view nominees, and cast their votes online, while administrators can manage the entire election process through a dashboard.

The user manual includes the following sections:

- System Requirements
- Software Installation
- Running the System
- Using the Application
- System Controls

7.1 System Requirements

Before installing the Online Voting System, the user must ensure that the system meets the minimum hardware and software requirements.

Hardware Requirements

The system requires the following hardware components:

- Processor: Intel Core i3 or higher
- RAM: Minimum 4 GB
- Storage: At least 100 GB free space
- Display: Standard monitor or laptop screen
- Internet Connection for accessing the system

Software Requirements

The following software tools are required to run the system:

- Operating System: Windows / Linux / macOS
- Programming Language: JavaScript
- Development Environment: Visual Studio Code

- Required Technologies:
- Node.js
- Next.js
- React.js

These technologies are required for frontend development, backend processing, and application execution.

7.2 Software Installation

The installation process involves setting up Node.js and installing required project dependencies.

Step 1 – Install Node.js

Download Node.js from the official website:

<https://nodejs.org>

Install Node.js and ensure it is properly configured in the system.

Step 2 – Download Project Files

Copy or download the project folder which contains all necessary files such as:

- Frontend components
- Backend files
- Configuration files
- Database files

Ensure all files are placed in the same project directory.

Step 3 – Install Dependencies

Open the project folder in Visual Studio Code and run the following command in the terminal:

```
npm install
```

This command installs all required packages for the project.

7.3 Running the System

After installing all dependencies, the system can be started using the following steps:

1. Open the project folder in Visual Studio Code
2. Run the following command in the terminal:

npm run dev

3. Open a web browser and go to:

http://localhost:3000

The Online Voting System interface will be displayed.

Users can now log in and access the system.

7.4 Using the Application

The system provides different interfaces for voters and administrators.

Voter Side

- Login using registered credentials
- View list of nominees
- Select a candidate and cast vote
- Receive confirmation after voting

Once the vote is cast, the voter cannot vote again.

Admin Side

- Login to admin dashboard
- Add or manage nominees
- View registered voters
- Monitor votes
- View and publish results

The admin dashboard provides full control over the election process.

7.5 System Controls

The Online Voting System includes basic controls for smooth operation:

- Login authentication to ensure secure access
- One-time voting restriction for each voter
- Real-time vote storage and result calculation
- Secure data handling to prevent unauthorized access

These controls ensure that the voting process is safe, accurate, and transparent.

8.CONCLUSION:

8.1 Summary of the Project

The **Online Voting System** developed in this project aims to simplify and modernize the election process by providing a secure and efficient digital platform for voting.

Traditional voting methods often require significant time, manpower, and resources, and may be prone to errors and delays. The proposed system addresses these challenges by enabling online voting through a web-based application.

The system allows registered voters to log in securely and cast their votes through an easy-to-use interface. It ensures that each voter can vote only once by implementing authentication mechanisms and vote validation techniques. This helps maintain the integrity and fairness of the election process.

The project includes several modules such as voter registration, nominee management, vote casting, vote storage, and result calculation. The administrator can manage the entire election process through a dashboard, where they can add candidates, monitor voting activity, and publish results.

The system uses modern web technologies to provide a responsive and interactive user experience. All votes are stored securely in the database, and the results are calculated automatically, reducing manual effort and minimizing errors.

Additionally, the system ensures transparency and accuracy by maintaining proper records of votes and preventing unauthorized access. The testing phase confirmed that the system performs efficiently and handles user operations smoothly.

Overall, the Online Voting System provides a reliable, fast, and user-friendly solution for conducting elections in institutions and organizations. The project demonstrates how digital technology can enhance the efficiency, security, and transparency of the voting process.

8.2 Future Enhancements

Although the Online Voting System performs effectively, several enhancements can be implemented to improve its functionality and scalability.

One possible enhancement is the integration of advanced authentication methods such as

One-Time Password (OTP) or biometric verification to further strengthen system security and prevent unauthorized access.

Another improvement would be the implementation of blockchain technology to ensure complete transparency and tamper-proof vote storage. This would increase trust in the election process.

The system can also be enhanced by developing a mobile application version, allowing users to vote easily through smartphones. This would improve accessibility and user convenience. Additionally, the system can be scaled to support large-scale elections with a higher number of users and improved performance optimization.

Advanced analytics and reporting features can also be added to provide detailed insights into voting patterns and election statistics.

With these enhancements, the Online Voting System can become a more powerful and secure platform suitable for large-scale and real-world election scenarios.

BIBLIOGRAPHY

Book References

1. Kogent Learning Solutions Inc. – *Web Technologies: HTML, JavaScript, PHP, Java, JSP, XML and AJAX*
2. Achyut S. Godbole & Atul Kahate – *Web Technologies*
3. E. Balagurusamy – *Programming with JavaScript*

Website References

1. <https://react.dev>
2. <https://nextjs.org>
3. <https://nodejs.org>
4. <https://developer.mozilla.org>
5. <https://www.w3schools.com>

APPENDIX

(a) Program Coding

1. Dashboard:

```
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import { getToken } from "../../utils/auth";

export default function Dashboard() {
  const router = useRouter();
  const [stats, setStats] = useState({});

  useEffect(() => {
    const token = getToken();

    if (!token) {
      router.push("/admin/login");
      return;
    }

    fetch("/api/admin/stats")
      .then(res => res.json())
      .then(data => setStats(data.stats));
  }, []);

  return (
    <div>
      <h1>Admin Dashboard</h1>
      <p>Total Voters: {stats.voters}</p>
      <p>Total Votes: {stats.votesCast}</p>
    </div>
  );
}
```

2. Voting Page:

```
"use client";

import { useState, useEffect } from "react";

export default function Voting() {
  const [voterId, setVoterId] = useState("");
  const [nominees, setNominees] = useState([]);
  const [selected, setSelected] = useState(null);

  useEffect(() => {
    fetch("/api/nominees")
```

```

    .then(res => res.json())
    .then(data => setNominees(data.data));
  }, []);

const submitVote = async () => {
  await fetch("/api/vote", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      voterId,
      nomineeId: selected
    })
  });
};

alert("Vote Submitted");
};

return (
  <div>
    <h1>Vote</h1>

    <input
      placeholder="Enter Voter ID"
      onChange={(e) => setVoterId(e.target.value)}
    />

    {nominees.map(n => (
      <div key={n._id} onClick={() => setSelected(n._id)}>
        {n.name}
      </div>
    ))}

    <button onClick={submitVote}>Submit Vote</button>
  </div>
);
}

```

3. Fetch Votes:

```

"use client";

import { useEffect, useState } from "react";

export default function Votes() {
  const [votes, setVotes] = useState([]);

  useEffect(() => {
    fetch("/api/admin/fetch-votes")
      .then(res => res.json())
      .then(data => setVotes(data.votes));
  });
}

```

```

}, []);

return (
  <div>
    <h1>Votes List</h1>

    {votes.map(v => (
      <div key={v._id}>
        <p>{v.voter.name}</p>
        <p>{v.nominee.name}</p>
      </div>
    ))}
  </div>
);
}

```

4. User Authentication and Login:

```

"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";

export default function AdminLogin() {
  const router = useRouter();

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleLogin = async () => {
    try {
      const res = await fetch("/api/admin/login", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({ email, password })
      });

      const data = await res.json();

      if (res.ok) {
        localStorage.setItem("token", data.token);
        router.push("/admin/dashboard");
      } else {
        alert(data.message);
      }
    } catch (err) {
      console.error(err);
      alert("Server Error");
    }
  };
}

```

```

return (
  <div>
    <h2>Admin Login</h2>

    <input
      type="email"
      placeholder="Enter Email"
      onChange={(e) => setEmail(e.target.value)}
    />

    <input
      type="password"
      placeholder="Enter Password"
      onChange={(e) => setPassword(e.target.value)}
    />

    <button onClick={handleLogin}>Login</button>
  </div>
);
}

```

5.Admin Dashboard:

```

"use client";

import { useEffect, useState } from "react";

export default function Dashboard() {
  const [stats, setStats] = useState({
    voters: 0,
    nominees: 0,
    votes: 0
  });

  useEffect(() => {
    fetch("/api/admin/stats")
      .then((res) => res.json())
      .then((data) => setStats(data.stats))
      .catch((err) => console.log(err));
  }, []);

  return (
    <div>
      <h1>Admin Dashboard</h1>

      <div>
        <p>Total Voters: {stats.voters}</p>
        <p>Total Nominees: {stats.nominees}</p>
        <p>Total Votes: {stats.votes}</p>
      </div>
    </div>
  );
}

```

```
);  
}
```

6. Server Setup:

```
const express = require("express");  
const mongoose = require("mongoose");  
const cors = require("cors");  
  
const app = express();  
app.use(express.json());  
app.use(cors());  
  
mongoose.connect("mongodb://127.0.0.1:27017/votingDB")  
  .then(() => console.log("Database Connected"))  
  .catch(err => console.log(err));  
  
app.listen(5000, () => {  
  console.log("Server running on port 5000");  
});
```

7. Voter module:

```
const mongoose = require("mongoose");  
  
const voterSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  voterId: String,  
  password: String,  
  hasVoted: {  
    type: Boolean,  
    default: false  
  }  
});  
  
module.exports = mongoose.model("Voter", voterSchema);
```

8. Get Nominee API:

```
const express = require("express");  
const router = express.Router();  
const Nominee = require("../models/Nominee");  
  
router.get("/nominees", async (req, res) => {  
  const nominees = await Nominee.find();  
  res.json({ success: true, data: nominees });  
});  
  
" });  
});
```


9. Vote Casting API:

```
const Vote = require("./models/Vote");
const Voter = require("./models/Voter");
const Nominee = require("./models/Nominee");

router.post("/vote", async (req, res) => {
  const { voterId, nomineeId } = req.body;

  const voter = await Voter.findOne({ voterId });

  if (!voter) {
    return res.status(400).json({ message: "Voter not found" });
  }

  if (voter.hasVoted) {
    return res.status(400).json({ message: "Already voted" });
  }

  await Vote.create({ voterId, nomineeId });

  await Nominee.findByIdAndUpdate(nomineeId, {
    $inc: { voteCount: 1 }
  });

  voter.hasVoted = true;
  await voter.save();

  res.json({ success: true, message: "Vote recorded" });
});
```

10. Dashboard Stats API:

```
router.get("/admin/stats", async (req, res) => {
  const voters = await Voter.countDocuments();
  const nominees = await Nominee.countDocuments();
  const votes = await Vote.countDocuments();

  res.json({
    stats: {
      voters,
      nominees,
      votesCast: votes
    }
  });
});
```